23/10/2026

Report

Intelligent Autonomous Robotics Systems

Presented By:
BENJAMIN DARIUS MONTAGUE

Table of Contents

Introduction 2
Methodology ..... 3 - 10
Results and Evaluation ..... 10 - 25
Discussion and Ethical Considerations ..... 25 - 33
Conclusion and Recommendations ..... 33 - 35
Bibliography ..... 35 - 37

Introduction

Autonomous mobile robot navigation heavily revolves around enabling robots to perceive their environment, plan suitable paths safely and accordingly and even control their movement in order to reach goal destinations or targets while also still avoiding obstacles. To ensure effective navigation procedures, any form of perception used within the environment needs to be accurate, any path-planning algorithms needs to be efficient and any control mechanisms also need to be safe and reliable.

Component A focuses on an autonomous mobile robot simulation implemented in Webots. The system integrates LiDAR and Camera-based perception, Dijkstra's and A* path-planning algorithms, and a PID controller to support obstacle assessment, planning of routes and even waypoint tracking. Performance of the robot is evaluated using the length of the path, nodes explored, navigation time, final goal error and route deviation. Component A also examines the effects of tuning the PID gains on the performance of the robot. The limitations behind deploying autonomous navigation in real-world environments are also considered.

Autonomous systems also involve multiple robots which work together to perform a collective objective through means of coordination, communication and even independent decision-making. Such systems can help to improve task allocation, navigation and operational efficiency in a variety of environments which consist of obstacles and multiple robots.However, achieving effective coordination requires for suitable planning algorithms, communication techniques, and even reliable decision-making methods.

Component B focuses on a multi-robot simulation implemented in Webots using three autonomous robots. The system then integrates consensus algorithms, Particle Swarm Optimization (PSO), A* path planning and a neural network to support coordination, waypoint allocation and even navigation decisions. Performance metrics tied to this system are evaluated through consensus convergence, PSO optimization results, neural network classification metrics and even the successful completion of all assigned navigation routes. Additionally, this component focuses on the limitations, safety risks, and ethical implications tied to deploying autonomous multi-robot systems.

Methodology

Component A

Sensor Simulation and Perception Pipeline

The perception pipeline was implemented in Webots using simulated LiDAR and Camera Sensors set up on top of the robot. LiDAR readings were processed to filter invalid values and only extract the minimum, front, left and right distance measurements for obstacle detection purposes.

Camera images were also processed using OpenCV, where the images were converted from BGRA to BGR format, followed by another conversion to Grayscale and Canny Edge detection. Edge pixels were also counted within selected image regions in order to provide visual evidence of potential obstacles.

A rule-based sensor fusion approach was also implemented to help combine LiDAR distance measurements with camera-based edge information. LiDAR provided proximity data, whilst the camera supplied visual feature information. Threshold-based rules were also used to interpret these inputs and support the controller's obstacle assessment and navigation decisions.

Put together, each of these techniques were selected due to their ease with implementing them in Python, being easily integrated in the Webots controller, and also being lightweight and not burdensome to the resources of the computer. Conversion of images into Grayscale can help to reduce any image-processing complexity, and the use of Canny Edge detection can also help to extract visual boundaries without needing an object-detection model that has been trained for such purposes. Combining LiDAR and camera data provides complementary information from two sensor modalities.

The sensor-based fusion approach has limitations since edge detection doesn't necessarily identify objects semantically or directly measure their distance. Additionally, threshold-based sensor fusion becomes very sensitive to errors found within the sensor and to the thresholds and rules used to interpret the readings, as well as the parameters used when designing these sensors.

Motion Planning and Path Finding

The robot's environment was represented as a 20 x 20 occupancy grid which covered a simulated Webots 6m x 6m grid arena. This gave each grid cell a resolution of around 0.3m. Additionally, the grid provided a discrete representation of the environment for path planning between the defined start and goal positions.

Four-directional movement was also used, with each horizontal or vertical transition being assigned a total cost of 1.

Dijkstra's algorithm and A* were implemented to calculate the path routes through the grid. Dijkstra's algorithm focuses on exploring reachable cells according to their accumulated path cost, whereas A* combines the accumulated cost, g(n), with a heuristic estimate, h(n), which calculates the estimated remaining distance to the goal. For this specific implementation, we used the Manhattan Distance heuristic since the grid supports four-directional movement. This heuristic ultimately estimates the remaining number of horizontal and vertical moves without overestimating the cost under the implemented movement model.

Both algorithms were evaluated using the same grid, start position and goal position. Each route's performance was compared using the path length, and the number of nodes explored to assess path optimality and computational efficiency. The resulting paths were visualized on the occupancy grid to compare their routes through the simulated environment.

Control System Design

A differential-drive kinematic model was implemented to describe and understand the robot's movement when navigating the grid. The model relates the left and right wheel angular velocities to the robot's linear and angular velocities. The simulated robot used a wheel radius of 0.08m and a wheel separation of 0.34m.

The linear velocity of the robot was calculated using the following equation:

$$v = (r / 2) \times (wR + wL)$$

The angular velocity of the robot was calculated using the following equation:

$$w = (r / L) \times (wR - wL)$$

where:

- v represents the robot's linear velocity measured in m/s.
- w represents the robots' angular velocity measured in rad/s.
- r represents the wheel radius which was 0.08m.
- L representes the distance between the two wheels which was 0.34m.

- wR and wL represent the angular velocities of the right and left wheels which are both measured in rad/s.

When both of the robot's wheels rotate at the same speed, the robot will end up moving in a straight line. When both of the robot's wheels rotate at different speeds, the robot also tends to change its direction. These equations provide the mathematical basis for controlling the robot's movement.

The robot's position and orientation change according to its linear and angular velocities. These relationships were represented through the following equations:

dx/dt = v x cos(theta)

dy/dt = v x sin(theta)

d[theta]/dt = w

where x and y represent the robot's position in the environment, theta represents its heading angle, v represents its linear velocity, and w represents its angular velocity. Together, these equations help to describe how the robot's wheel movements affect its position and orientation during navigation.

A Proportional-Integral-Derivative (PID) based controller was implemented to support waypoint tracking and the controller used position and heading information in order to calculate corrections to the robot's motion. This was represented using the following equation:

u(t) = Kp x e(t) + Ki x Integral e(t) dt + Kd x de(t)/dt

where:

- u(t) represents the controller output.
- e(t) represents the current error.
- Kp represents the proportional gain.
- Ki represents the integral gain.

- Kd represents the derivative gain.
- Integral e(t) dt represents the accumulated error over time.
- de(t)/dt represents the rate of change of the error.

The proportional term responded to the current error, and the integral terms accounted for any errors which were accumulated, whilst the derivative term responded to changes in error. Integral limiting and reset logic were also included to reduce excessive integral accumulation.

The calculated PID correction is used to adjust the relative speeds of the robot's wheels, allowing for it to change direction while following the planned path. In a differential-drive system, steering of the robot is achieved by changing the difference between the left and right wheel speeds. When both wheels rotate at similar speeds, the robot ends up moving straight, whereas different wheel speeds cause it to turn. The controller thus uses the calculated correction to reduce the robot's deviation from its desired path. Ultimately, the controller's performance depends on the selection of the PID gains. These gains were adjusted in separate experiments to understand their influence on the robot's position-error response.

The baseline controller used (K_p = 2.0), (K_i = 0) and (K_d = 0.2). All four configurations were tested under a base speed of 1.0 on both wheels. Three separate experiments were conducted by adjusting these gain values in order to investigate their effect on the position-error response.

Configuration	**Kp**	**Ki**	**Kd**
Baseline	2.0	0	0.2
Experiment 1	3.0	0	0.2
Experiment 2	3.0	0.05	0.2
Experiment 3	3.0	0.05	0.4

Experiment 1 involves increasing the proportional gain to 3.0; Experiment 2 added an integral gain of 0.05; and Experiment 3 increased the derivative gain to 0.4 whilst retaining the other two Experiment conditions. The resulting position-error, heading-error, and PID-correction data were recorded and plotted to compare controller performance. The experiments were also used to evaluate how changes to the PID gains influence the robot's control response during navigation.

Controller performance was evaluated using the recorded position-error, heading-error, and PID-correction responses over time. Each of the response curves were used to investigate the effects of gain tuning on the robot's tracking performance. Factors such as overshoot, rise time, settling time, and steady-state error were considered to assess the response behind each experiment.

System Integration

The perception, planning and control modules were integrated into a single closed-loop Webots simulation, which enabled the robot to receive simulated sensor readings and position information. It also enabled the robot to use the planning and waypoint information in order to guide navigation and even helped enable the robot to adjust its movement through the controller. Obstacle-assessment logic helped to support navigation decisions made during execution.

The robot's actual trajectory path and controller response data were recorded for evaluation. Planned and actual movement were also compared, and performance was assessed using distance travelled, time to reach the goal, final goal error, and even route deviation. This enabled the evaluation process to consider the robot's behavior when executing the navigational task.

Component B

Multi-Robot Simulation Environment and Architecture

This environment was developed using Webots which contained a world file that consisted of a rectangular arena, obstacles and three autonomous robots. Each robot found within this environment was controlled by a single Python Controller that enabled them to navigate the environment and participate in the shared task. The simulation helped to provide a controlled environment which collected information relating to the behavior and coordination of all 3 robots as they navigated around the arena.

Agent Roles and Cooperative Objective

The system consisted of three autonomous robots, each identified as robot_1, robot_2, and robot_3. All three robots performed the same fundamental task of navigating their environment by being assigned a sequence or route of target waypoints that they were supposed to visit. Although their roles are identical, they each followed their own assigned route. The cooperative objective was for all three robots to navigate the environment, reach their assigned targets respectively, and coordinate their movements whilst still operating within the same shared space.

Communication Topology

In terms of communication, the robots utilize Webots Emitter and Receiver devices in order to exchange information. This enabled the agents to share relevant information used by the coordination system. Communication devices assisted in supporting the exchange of information and coordination between robots whilst executing tasks.

Justification of Environment Design and Controller Structure

The simulation heavily relied on Python-based robot controllers in order to manage navigation, communication, and coordination. GPS and Inertial Measurement Units (IMU) devices were used to provide data which was used to obtain information about each robot's position and orientation. A* path planning was also used to generate routes around static obstacles which were already known, whilst the coordination mechanisms aided in task allocation and interaction between agents. Ultimately, this structure separated navigation, communication and all coordination functions, which made the system easier to develop, test and evaluate. The use of static obstacles and a rectangular arena made the process repeatable when conducting tests to determine how the robots behave during simulation runs and conduct evaluations based on their behavior.

Average Consensus Algorithm

An average consensus algorithm was implemented to enable the three autonomous robots to converge towards a common state value through iterative information exchange. Each robot maintains a consensus value and communicates with other agents using Webots Emitter and Receiver devices. By receiving successful consensus updates, the robots were able to adjust each of their values towards a shared result. This approach heavily influenced how the robots distributed and coordinated their tasks since the mechanism enabled them to reach an agreement after sharing their values rather than having to rely on central controller to communicate their tasks.

Particle Swarm Optimization

PSO was implemented to optimize the allocation and ordering of target waypoints amongst the three robots. The algorithm evaluates the candidate waypoint assignments through the use of a fitness function which takes into account the route-related costs and constraints. Through successful iterations, particles are able to update their candidate solutions using information about their own best solutions and the best solution discovered by the swarm. The resulting waypoint assignments are then utilized with A* path planning to generate navigation routes around the Webots environment.

Cooperative Behavior and Convergence

The consensus algorithm supports agreement between the robots, whilst PSO supports the distribution and organization of the waypoint-visiting task. Together, these methodologies provide a basis for coordinated multi-robot behavior. Their performance is evaluated using consensus-value convergence, PSO fitness progression and the robot's ability to execute their assigned waypoint routes in the Webots environment collision-free.

Results and Evaluation

Component A

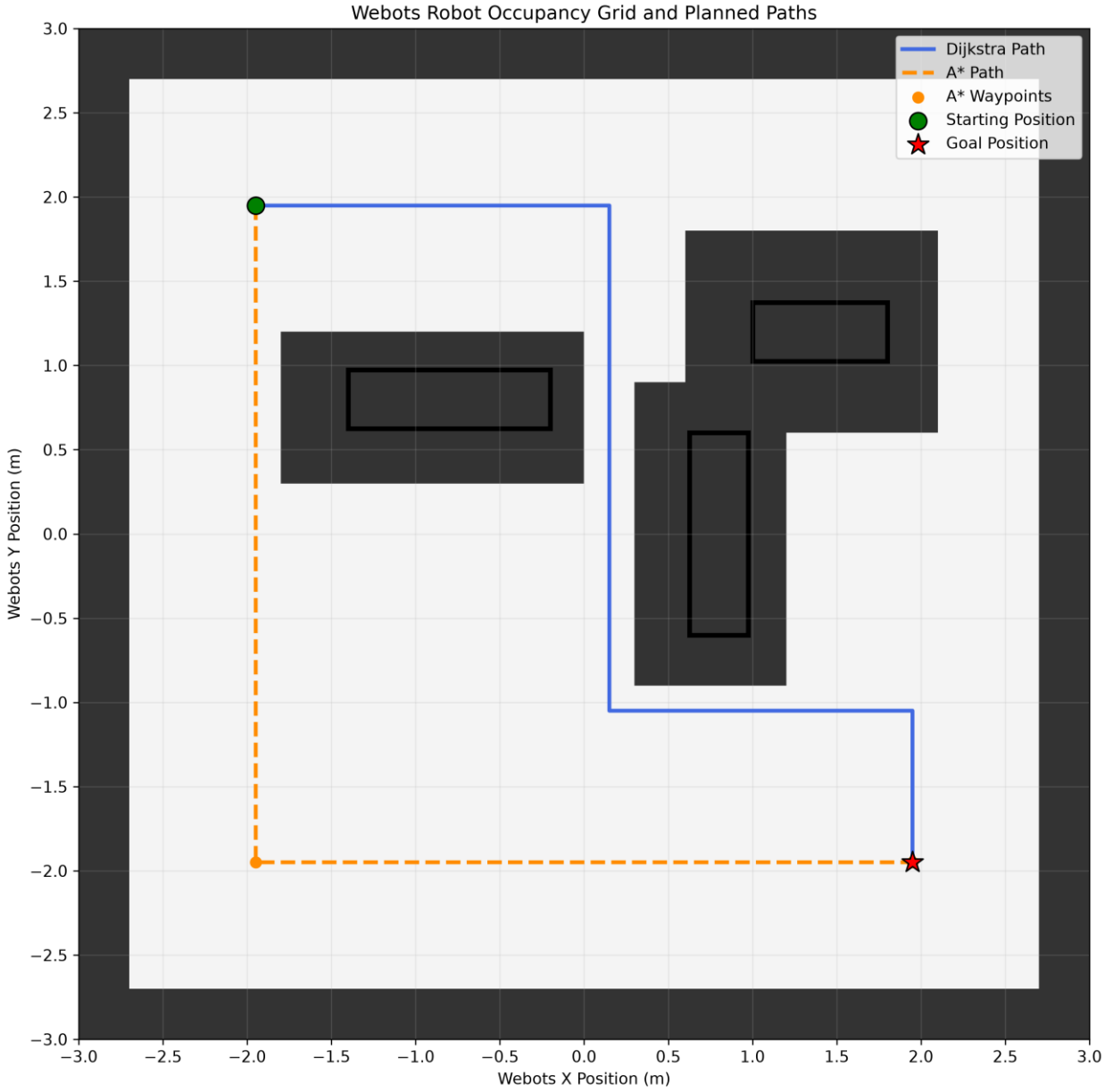
Motion Planning and Path Finding

The performance of Dijkstra's Algorithm and A* was evaluated using the same 20 x 20 occupancy grid, starting position and goal position. Both algorithms generated a path with a length of 7.80m, demonstrating equivalent path-length performance in the tested environment. However, the number of nodes explored differed between the two algorithms.

Performance Metric	**Djikstra's Algorithm**	**A* Algorithm**
Path Length	7.80m	7.80m
Nodes Explored	258	127
Relative Search Efficiency	Baseline	50.8% fewer nodes explored

A* ended up exploring 127 nodes compared with 258 from Dijkstra's algorithm, which represented a reduction of around 50.8% in the number of nodes explored. This improvement ultimately is due to the Manhattan Distance heuristic, since it guided the search towards the goal instead of blindly exploring all nodes solely based on the accumulated path cost.

Despite having explored fewer nodes, A* produced the same path length as Dijkstra's algorithm, which indicates that for the tested environment, A* achieved the same path length result with a much smaller search effort. Dijkstra's algorithm still remains useful as a baseline algorithm since it explores all nodes to determine the shortest path based on the accumulated cost rather than relying on a heuristic.

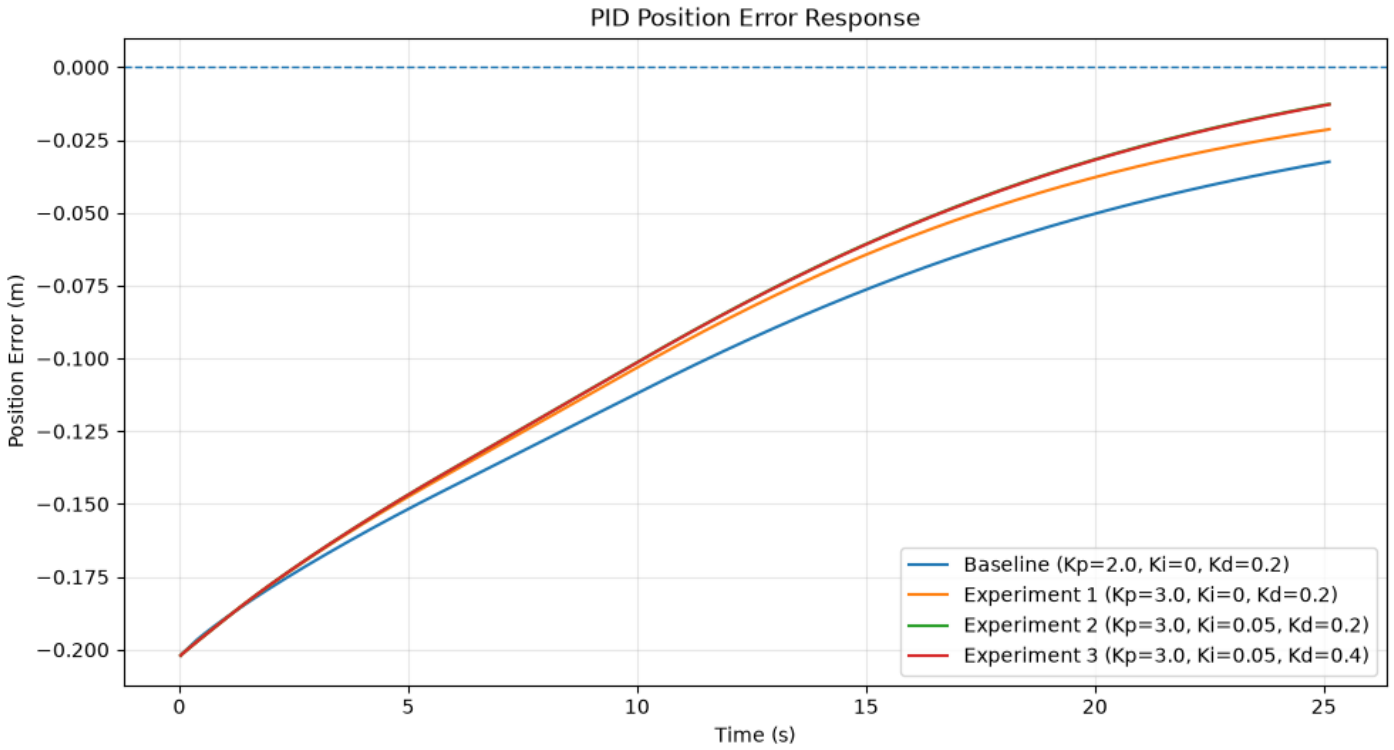


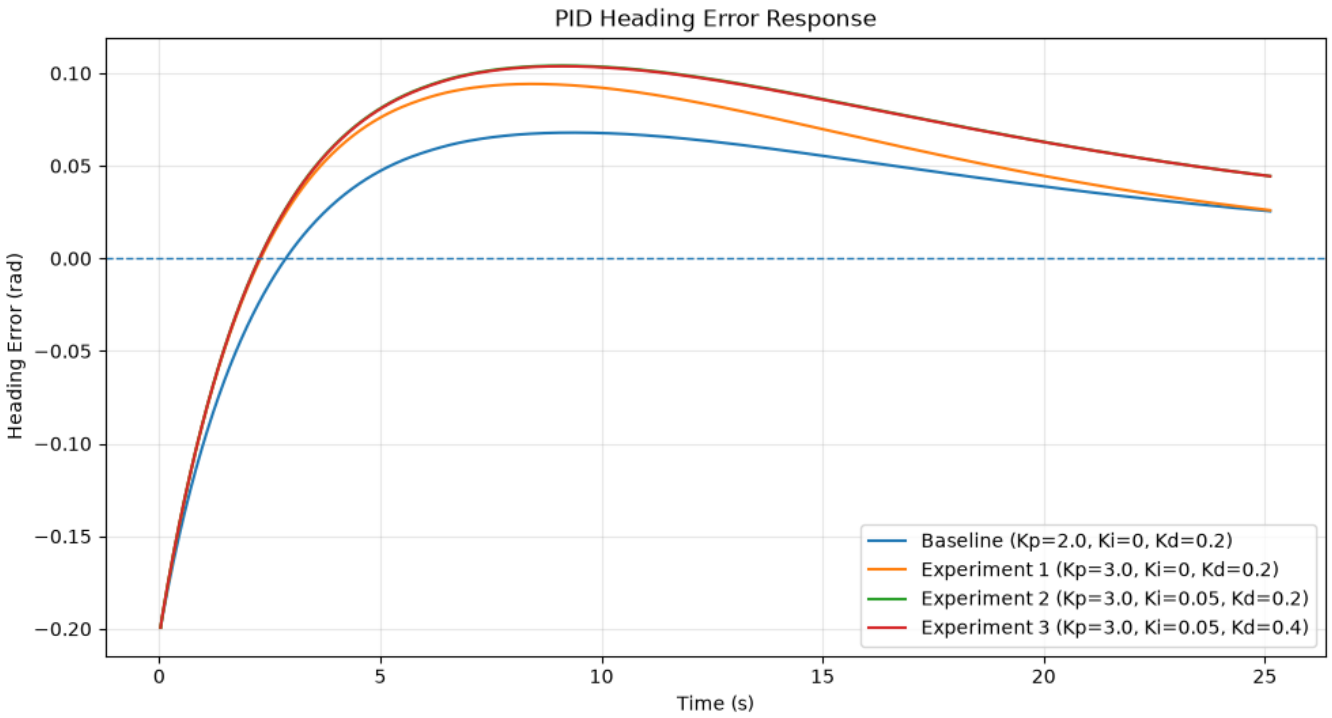
The occupancy grid visualization illustrated the routes generated by both algorithms in accordance with their starting positions, goal positions and obstacles. The figure is used to examine which of the paths avoided occupied cells and how the algorithms navigated around obstacles. Together, the numerical comparison and visualization demonstrated that A* reduced the search effort whilst still maintaining the same path length as Dijkstra's algorithm in this experiment.

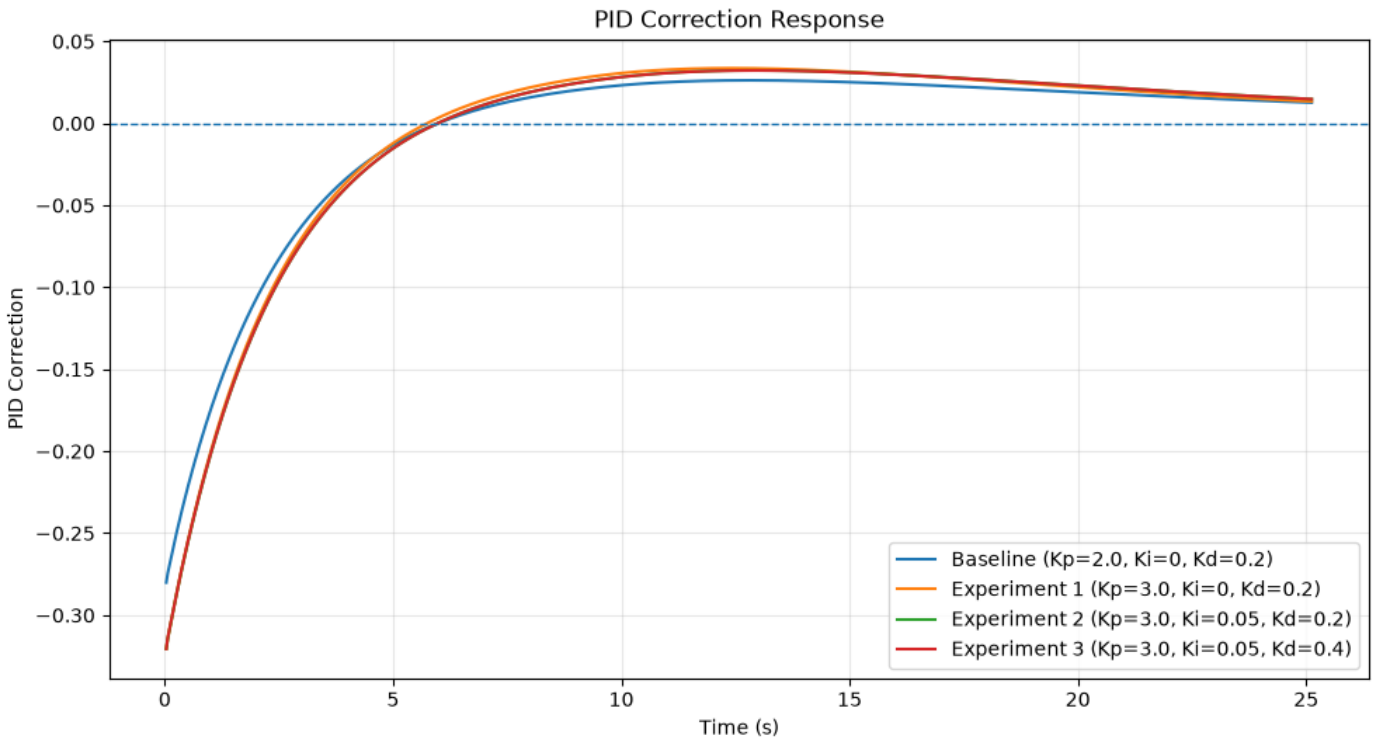
Control System Design

The PID Controller was evaluated using four gain configurations in order to assess the effects of tuning on the different gains relative to the path and its tracking performance.

Configuration	**Kp**	**Ki**	**Kd**
Baseline	2.0	0	0.2
Experiment 1	3.0	0	0.2
Experiment 2	3.0	0.05	0.2
Experiment 3	3.0	0.05	0.4








```
================================================================================================
PID PERFORMANCE ANALYSIS
================================================================================================
                          Experiment  Overshoot Proxy (rad)  Rise-Time Proxy (s)  Position Settling Time (s)  Heading Settling Time (s)  Position Steady-State Error (m)
      Baseline (Kp=2.0, Ki=0, Kd=0.2)               0.199305                  NaN                         NaN                         NaN                       -0.036184
   Experiment 1 (Kp=3.0, Ki=0, Kd=0.2)               0.199305                  NaN                         NaN                         NaN                       -0.024617
Experiment 2 (Kp=3.0, Ki=0.05, Kd=0.2)               0.199305               21.152                         NaN                         NaN                       -0.016526
Experiment 3 (Kp=3.0, Ki=0.05, Kd=0.4)               0.199305               21.216                         NaN                         NaN                       -0.016699
```

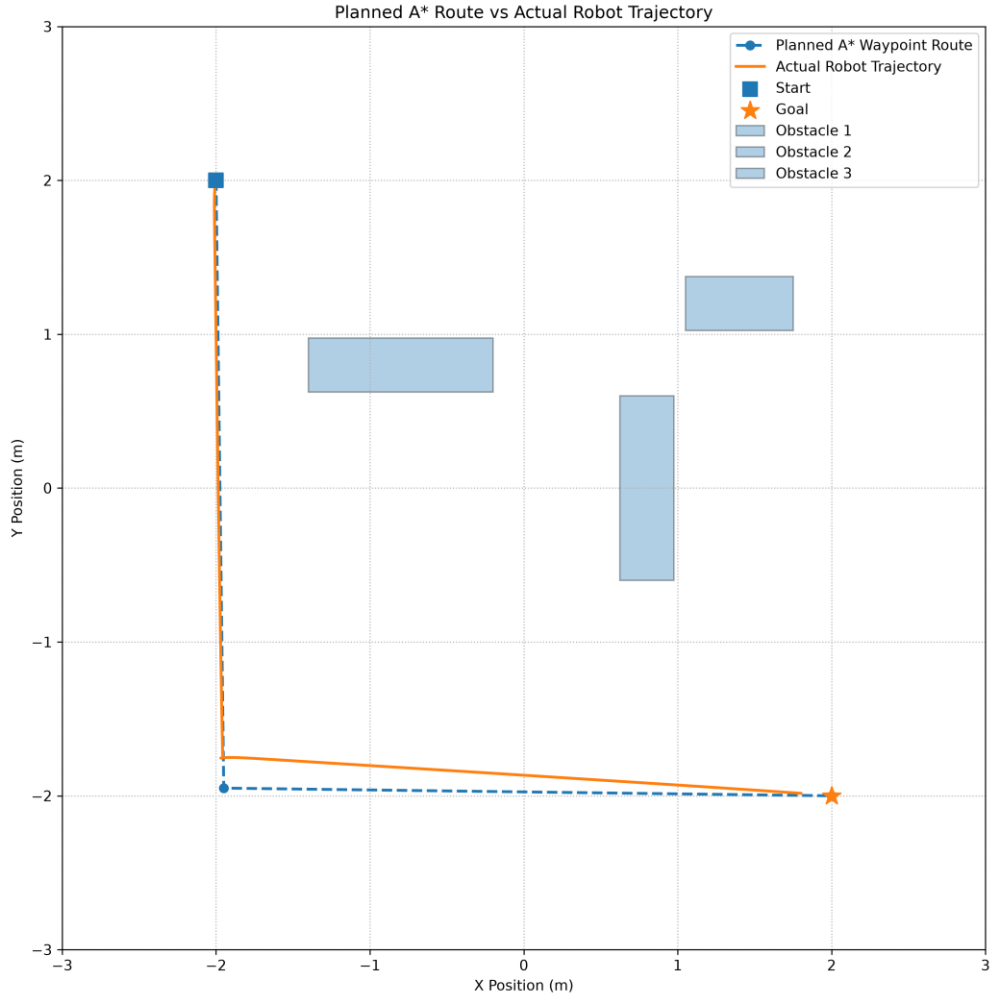
Heading Steady-State Error (rad)
0.028340
0.029834
0.048364
0.048325

Increasing the Kp reduced the mean absolute position error in the final 10% of the recording from 0.0362m to 0.0246m. Adding Ki reduced it further to 0.0165m, whilst increasing Kd to 0.4 results in no additional improvement. However, the mean absolute heading error increased from 0.0283 rad in the baseline experiment through to 0.0483 rad in Experiments 2 and 3, displaying a trade-off between position and heading accuracy.

The response curves for position error, heading error, and PID correction illustrate how each gain configuration changes the controller's response levels over time. The reduction in position-error percentage from 90% to 10% over the span of 21.152 seconds was for Experiment 2 and 21.216 seconds for Experiment 3. The other configurations couldn't yield a valid value through this calculation. Additionally, none of the configurations reached the specified settling bands of +/- 0.01m and +/- 0.0.1 rads before the recording ended. The peak absolute heading error was 0.199 rad in all four experiments, which represented a peak-error measure, and not a step-response overshoot.

Ultimately, the tuning improve position accuracy, but the settling time and stability of the controller cannot be established from these metrics alone. Each of the response curves should be used to check for growing errors. Experiment 2 showcased the best results, whilst Experiment 3 provided no meaningful improvement with the high Kd gain.

Trajectory Plot



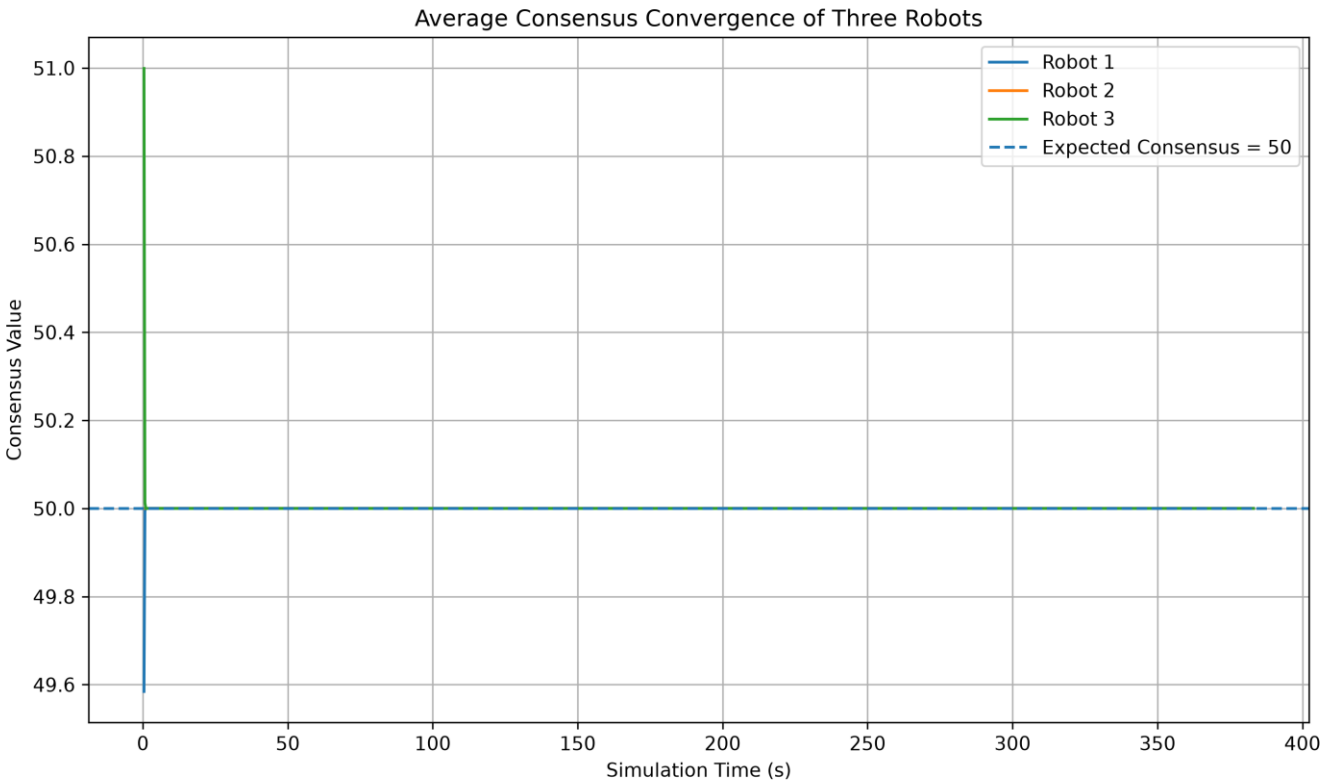
In this image, it can be seen the comparison between the planned A* route with the actual trajectory covered by the robot in Webots. The robot followed the planned route closely, although it deviated at the next waypoint before approaching the goal.

Performance Metric	**Result**
Planned Path Length	7.80 m
Actual Distance Travelled	7.55 m
Time to Reach Goal Region	98.272 s
Final Goal Error	0.199 m
Mean Route Deviation	0.056 m
Maximum Route Deviation	0.196 m

Overall, these results demonstrate that the integrated navigation system successfully reached the goal whilst maintaining reasonably accurate path tracking in the simulated environment.

Component B

Consensus Algorithm

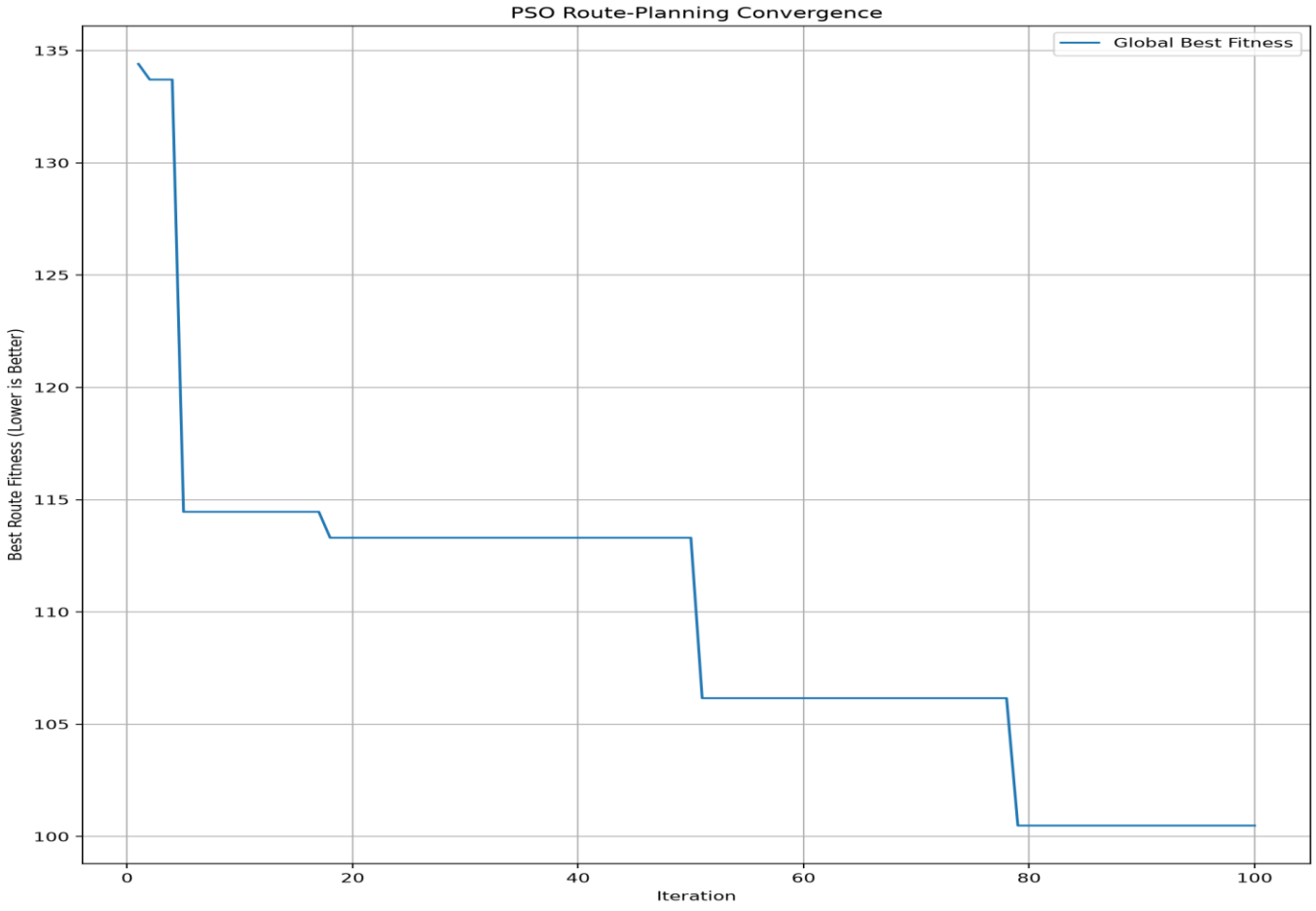



```
time,robot_1,robot_2,robot_3
0.384,49.584448352,50.0,50.9990192
0.704,49.994825378350086,50.0,50.01244021506411
1.024,49.99993556346701,50.0,50.00015491071282
1.344,49.99999919760959,50.0,50.000001929012384
1.6640000000000001,49.9999999900083,50.0,50.00000002402086
1.984,49.999999999875584,50.0,50.00000000029912
2.3040000000000003,49.99999999999845,50.0,50.00000000000372
2.624,49.99999999999998,50.0,50.00000000000004
2.944,49.99999999999999,50.0,50.00000000000001
3.2640000000000002,49.99999999999999,50.0,50.00000000000001
3.584,49.99999999999999,50.0,50.00000000000001
3.904,49.99999999999999,50.0,50.00000000000001
4.224,49.99999999999999,50.0,50.00000000000001
4.5440000000000005,49.99999999999999,50.0,50.00000000000001
4.864,49.99999999999999,50.0,50.00000000000001
5.184,49.99999999999999,50.0,50.00000000000001
```

With regards to the Consensus Algorithm and Convergence methodology used for Component B, the multi-robot system that was implemented utilized an Average-Consensus Mechanism which enabled the three robots

to converge towards a common state value through updates which were rolled out in an iterative manner. Analysis conducted on the latest simulation run of the Webots environment revealed the following, 1197 samples were recorded, and for each of these samples, the consensus values developed and approached a value of 50. The recorded consensus error decreased from approximately 0.5364 at around 0.384 seconds to effectively 0 after 2.624 seconds. This demonstration showcased rapid convergence in the recorded simulation data. Additionally, the final value demonstrated that it was consistent with the arithmetic mean of the initial robot values (20, 50 and even 80). However, since the first recorded sample had already contained values which were close to 50, the transition from the original initial conditions has to be verified separately before being claimed as directly demonstrated by these logs.

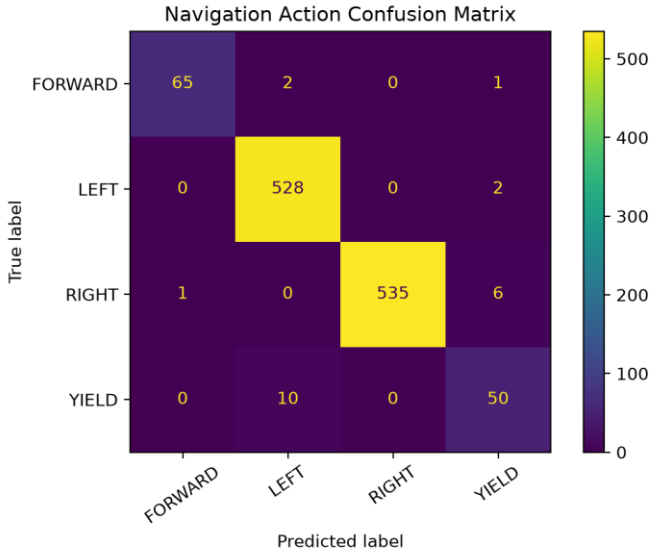
Particle Swarm Optimization



iteration	best_route_fitness
1	134.40554236968316
2	133.70859807251554
3	133.70859807251554
4	133.70859807251554
5	114.46022526263917
6	114.46022526263917
7	114.46022526263917
8	114.46022526263917
9	114.46022526263917
10	114.46022526263917
11	114.46022526263917
12	114.46022526263917
13	114.46022526263917
14	114.46022526263917
15	114.46022526263917
16	114.46022526263917
17	114.46022526263917
18	113.30570476459206
19	113.30570476459206
20	113.30570476459206
21	113.30570476459206
22	113.30570476459206
23	113.30570476459206
24	113.30570476459206
25	113.30570476459206
71	106.16184009228583
72	106.16184009228583
73	106.16184009228583
74	106.16184009228583
75	106.16184009228583
76	106.16184009228583
77	106.16184009228583
78	106.16184009228583
79	100.48722985664237
80	100.48722985664237
81	100.48722985664237
82	100.48722985664237
83	100.48722985664237
84	100.48722985664237
85	100.48722985664237
86	100.48722985664237
87	100.48722985664237
88	100.48722985664237
89	100.48722985664237
90	100.48722985664237
91	100.48722985664237
92	100.48722985664237
93	100.48722985664237
94	100.48722985664237
95	100.48722985664237
96	100.48722985664237
97	100.48722985664237
98	100.48722985664237
99	100.48722985664237
100	100.48722985664237

Particle Swarm Optimization (PSO) was utilized in Component B's Webots environment to optimize the allocation and ordering of target waypoints amongst the three robots. The algorithm evaluated candidate solutions across more than 100 iterations, and recorded the global best fitness attached to each iteration. The best fitness value decreased from 134.4055 in the first iteration down to 100.4872 in the final iteration. This ultimately represented a reduction percentage of around 25.24% in the optimization objective. Additionally, the convergence curve demonstrated several stages of improvement followed by periods where no best solution was found. The final fitness value remained unchanged at around iteration 79 and onwards. This indicated that the algorithm had stabilized during its final iterations. As a result of this, it demonstrated that the PSO objective showed improvement for the tested configuration. However, the fitness reduction shouldn't be interpreted as an equivalent reduction for the actual distance the robot travels. This is because the fitness function incorporates task allocation and even constraint related factors. Ultimately, further evaluation would be required across multiple runs in order to assess the consistency and flexibility of the optimization results.

Neural Network Results, Confusion Matrix and Classification Report




```
---Model Evaluation---
Training Examples: 4800
Test Examples: 1200
Test Accuracy: 0.9817

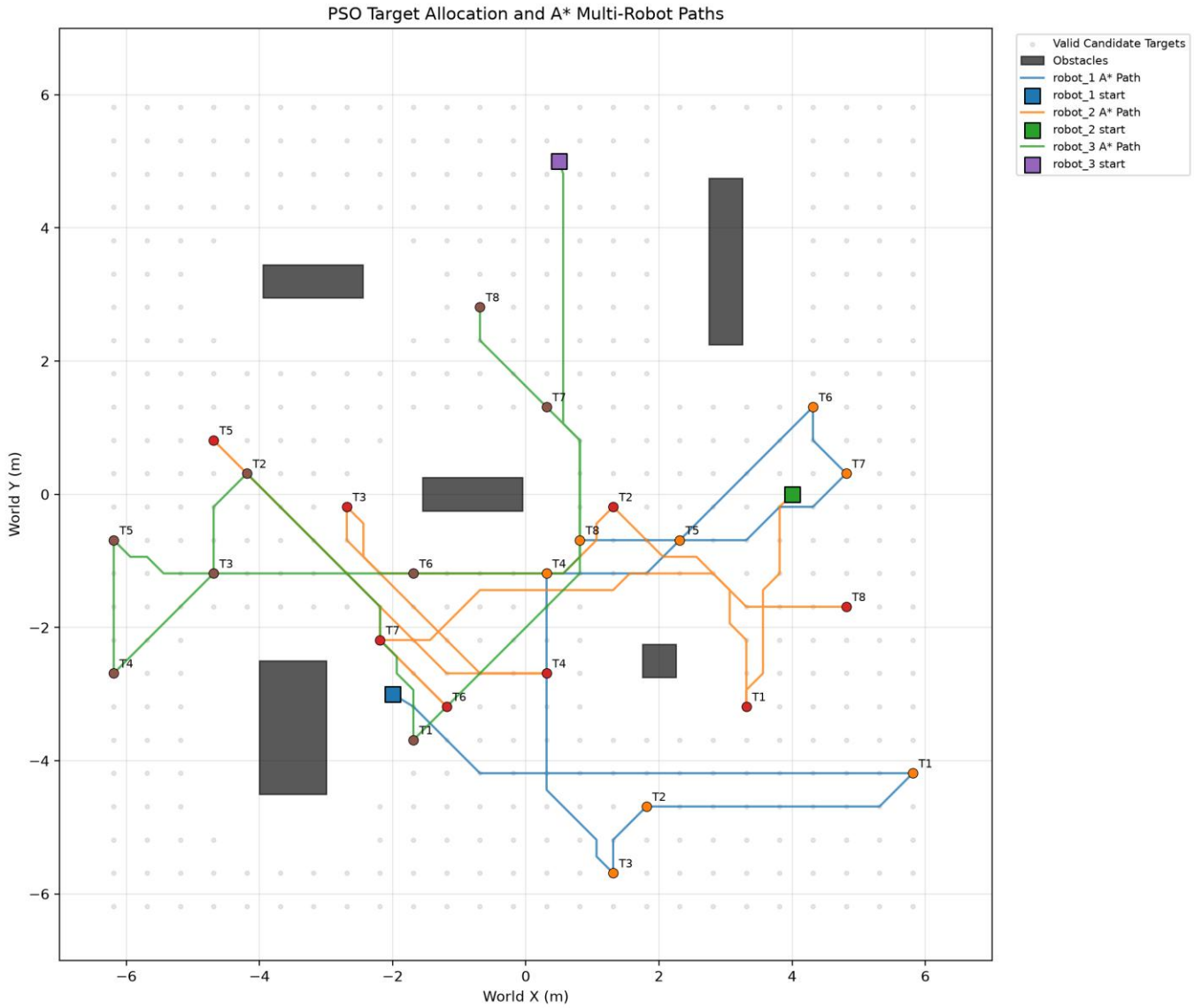
Classification Report:
              precision    recall  f1-score   support

     FORWARD       0.98      0.96      0.97        68
        LEFT       0.98      1.00      0.99       530
       RIGHT       1.00      0.99      0.99       542
       YIELD       0.85      0.83      0.84        60

    accuracy                           0.98      1200
   macro avg       0.95      0.94      0.95      1200
weighted avg       0.98      0.98      0.98      1200
```

In terms of Machine Learning, a neural network was implemented and trained on 6000 synthetically labelled samples in order to test and demonstrate decision-making. The model reserved 80% of the labelled samples for training and 20% of the labelled samples for testing. The results from this model produced a test accuracy of 98.17% across 1200 test samples when making predictions on the four navigation actions undertaken by the robot. These actions are FORWARD, LEFT, RIGHT, and YIELD. The classification report generated alongside the model showed strong favor towards FORWARD, LEFT, and RIGHT, each producing an F1-Score of 97.01%, 98,69%, and 99.35% respectively. However, the YIELD achieved a lower F1-Score of 84.03%, with 50 of the 60 true YIELD examples being correctly classified whilst the remaining 10 examples were predicted as LEFT. This weakness is very important since yielding helps to reduce any risks associated with collisions between robots sharing the same navigation space. Even though the results demonstrated that the model could reproduce the rule-generated action labels with a high accuracy value, it wasn't as useful since the dataset was synthetically generated. Hence, the test accuracy doesn't mirror real-world conditions or proves that the model can independently prevent collisions from occurring. Further evaluation of the Webots Navigation outcomes and additional safety checks is needed to ensure reliability when implemented in a real-world scenario.

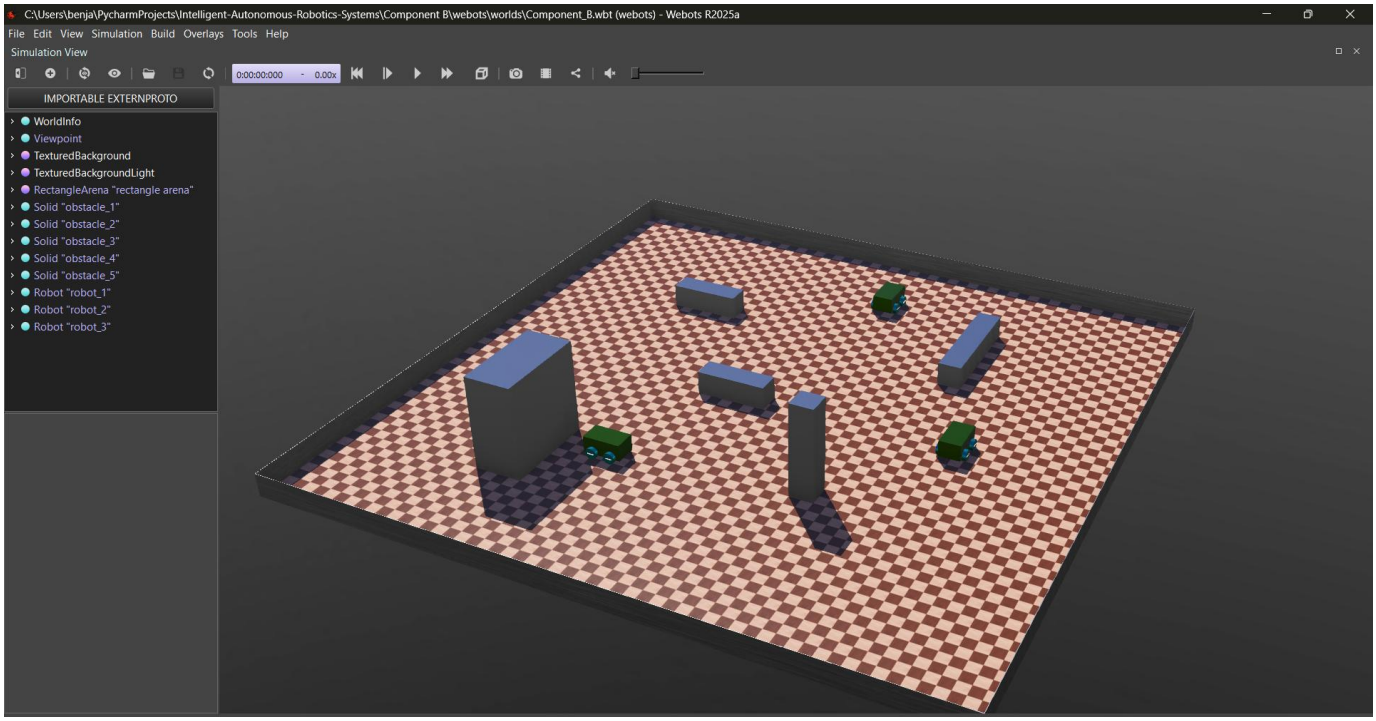
PSO and A Route Planning*



```
robot,sequence,target_x,target_y
robot_1,1,5.811410879802374,-4.188589120197626
robot_1,2,1.8114108798023736,-4.688589120197626
robot_1,3,1.3114108798023736,-5.688589120197626
robot_1,4,0.3114108798023736,-1.1885891201976264
robot_1,5,2.3114108798023736,-0.6885891201976264
robot_1,6,4.311410879802374,1.3114108798023736
robot_1,7,4.811410879802374,0.3114108798023736
robot_1,8,0.8114108798023736,-0.6885891201976264
robot_2,1,3.3114108798023736,-3.1885891201976264
robot_2,2,1.3114108798023736,-0.18858912019762641
robot_2,3,-2.6885891201976264,-0.18858912019762641
robot_2,4,0.3114108798023736,-2.6885891201976264
robot_2,5,-4.688589120197626,0.8114108798023736
robot_2,6,-1.1885891201976264,-3.1885891201976264
robot_2,7,-2.1885891201976264,-2.1885891201976264
robot_2,8,4.811410879802374,-1.6885891201976264
robot_3,1,-1.6885891201976264,-3.6885891201976264
robot_3,2,-4.188589120197626,0.3114108798023736
robot_3,3,-4.688589120197626,-1.1885891201976264
robot_3,4,-6.188589120197626,-2.6885891201976264
robot_3,5,-6.188589120197626,-0.6885891201976264
robot_3,6,-1.6885891201976264,-1.1885891201976264
robot_3,7,0.3114108798023736,1.3114108798023736
robot_3,8,-0.6885891201976264,2.8114108798023736
```

The PSO-based allocation assigned 24 target waypoints amongst the 3 robots, with each robot having a total of 8 waypoints to cover. The route visualization showed the assigned target sequences of the corresponding A* paths through an environment which contained static obstacles. Even though the target counts were balanced, the straight-line distances between consecutive targets do differ, by approximately 19.89m for Robot 1, 31.36m for Robot 2, and 19.95m for Robot 3. This excludes the initial start-to-target legs. With this in mind, it meant that equal target counts didn't result in equal travel workloads. The plotted routes also showcased shared navigation areas, highlighting the importance behind robot coordination and collision avoidance with obstacles and with other fellow robots. These results ultimately demonstrated the combination of PSO-based task allocation and A* route planning, but further measurements behind the actual travel distance, completion time, and even repeated run performance of each robot would be required in order to establish the overall efficiency tied to each robot and their objectives.

Webots Environment Simulation Setup and Test Results




```
robot_2: REACHED Waypoint 82/128 at (-4.19, 0.31) | Distance: 0.249 m
robot_3: REACHED Waypoint 110/114 at (-0.19, 1.81) | Distance: 0.248 m
robot_1 | Position=(1.36, -0.67) | Target=(1.06, -0.69) | Yaw=3.12 | Heading error=0.09 | Distance=0.30 | Left speed=1.76 | Right speed=2.00
robot_2 | Position=(-4.01, 0.39) | Target=(-3.94, 0.06) | Yaw=-1.45 | Heading error=0.09 | Distance=0.33 | Left speed=1.77 | Right speed=2.00
robot_3 | Position=(-0.03, 1.73) | Target=(-0.44, 2.06) | Yaw=2.50 | Heading error=-0.04 | Distance=0.53 | Left speed=2.00 | Right speed=1.89
robot_1: REACHED Waypoint 104/105 at (1.06, -0.69) | Distance: 0.247 m
robot_2: REACHED Waypoint 83/128 at (-3.94, 0.06) | Distance: 0.245 m
robot_3: REACHED Waypoint 111/114 at (-0.44, 2.06) | Distance: 0.248 m
robot_1: REACHED Waypoint 105/105 at (0.81, -0.69) | Distance: 0.245 m
robot_1: A* Route Completed (105/105 waypoints reached)
robot_2 | Action: LEFT | Nearest Robot: 4.08 m
robot_2 | Position=(-3.95, 0.10) | Target=(-3.69, -0.19) | Yaw=-1.23 | Heading error=0.40 | Distance=0.39 | Left speed=0.95 | Right speed=2.00
robot_3 | Action: FORWARD | Nearest Robot: 2.95 m
robot_3 | Position=(-0.32, 1.95) | Target=(-0.69, 2.31) | Yaw=2.47 | Heading error=-0.10 | Distance=0.52 | Left speed=2.00 | Right speed=1.74
robot_2: REACHED Waypoint 84/128 at (-3.69, -0.19) | Distance: 0.249 m
robot_3: REACHED Waypoint 112/114 at (-0.69, 2.31) | Distance: 0.247 m
robot_2 | Position=(-3.84, -0.14) | Target=(-3.44, -0.44) | Yaw=-0.99 | Heading error=0.36 | Distance=0.50 | Left speed=1.06 | Right speed=2.00
robot_3 | Position=(-0.56, 2.16) | Target=(-0.69, 2.56) | Yaw=2.36 | Heading error=-0.49 | Distance=0.42 | Left speed=2.00 | Right speed=0.70
robot_3: REACHED Waypoint 113/114 at (-0.69, 2.56) | Distance: 0.247 m
robot_2: REACHED Waypoint 85/128 at (-3.44, -0.44) | Distance: 0.250 m
robot_2 | Action: LEFT | Nearest Robot: 4.01 m
robot_2 | Position=(-3.66, -0.36) | Target=(-3.19, -0.69) | Yaw=-0.78 | Heading error=0.18 | Distance=0.58 | Left speed=1.51 | Right speed=2.00
robot_3 | Action: RIGHT | Nearest Robot: 3.50 m
robot_3 | Position=(-0.71, 2.35) | Target=(-0.69, 2.81) | Yaw=2.07 | Heading error=-0.56 | Distance=0.46 | Left speed=2.00 | Right speed=0.54
robot_2 | Position=(-3.42, -0.59) | Target=(-3.19, -0.69) | Yaw=-0.67 | Heading error=0.25 | Distance=0.25 | Left speed=1.34 | Right speed=2.00
robot_3 | Position=(-0.81, 2.58) | Target=(-0.69, 2.81) | Yaw=1.70 | Heading error=-0.62 | Distance=0.26 | Left speed=2.00 | Right speed=0.37
robot_2: REACHED Waypoint 86/128 at (-3.19, -0.69) | Distance: 0.245 m
robot_3: REACHED Waypoint 114/114 at (-0.69, 2.81) | Distance: 0.248 m
robot_3: A* Route Completed (114/114 waypoints reached)
```

```
robot_2: REACHED Waypoint 119/128 at (2.56, -1.19) | Distance: 0.248 m
robot_2: REACHED Waypoint 120/128 at (2.81, -1.19) | Distance: 0.246 m
robot_2 | Action: RIGHT | Nearest Robot: 1.59 m
robot_2 | Position=(2.58, -1.14) | Target=(3.06, -1.44) | Yaw=-0.07 | Heading error=-0.49 | Distance=0.57 | Left speed=2.00 | Right speed=0.72
robot_2 | Position=(2.84, -1.18) | Target=(3.06, -1.44) | Yaw=-0.39 | Heading error=-0.47 | Distance=0.34 | Left speed=2.00 | Right speed=0.77
robot_2: REACHED Waypoint 121/128 at (3.06, -1.44) | Distance: 0.250 m
robot_2 | Action: RIGHT | Nearest Robot: 2.11 m
robot_2 | Position=(3.07, -1.31) | Target=(3.31, -1.69) | Yaw=-0.64 | Heading error=-0.37 | Distance=0.45 | Left speed=2.00 | Right speed=1.03
robot_2: REACHED Waypoint 122/128 at (3.31, -1.69) | Distance: 0.247 m
robot_2 | Position=(3.29, -1.50) | Target=(3.56, -1.69) | Yaw=-0.79 | Heading error=0.20 | Distance=0.33 | Left speed=1.48 | Right speed=2.00
robot_2: REACHED Waypoint 123/128 at (3.56, -1.69) | Distance: 0.250 m
robot_2 | Action: LEFT | Nearest Robot: 2.64 m
robot_2 | Position=(3.49, -1.68) | Target=(3.81, -1.69) | Yaw=-0.56 | Heading error=0.53 | Distance=0.32 | Left speed=0.60 | Right speed=2.00
robot_2: REACHED Waypoint 124/128 at (3.81, -1.69) | Distance: 0.249 m
robot_2 | Position=(3.72, -1.78) | Target=(4.06, -1.69) | Yaw=-0.23 | Heading error=0.49 | Distance=0.36 | Left speed=0.71 | Right speed=2.00
robot_2: REACHED Waypoint 125/128 at (4.06, -1.69) | Distance: 0.248 m
robot_2 | Action: LEFT | Nearest Robot: 3.14 m
robot_2 | Position=(3.99, -1.81) | Target=(4.31, -1.69) | Yaw=0.07 | Heading error=0.28 | Distance=0.35 | Left speed=1.26 | Right speed=2.00
robot_2: REACHED Waypoint 126/128 at (4.31, -1.69) | Distance: 0.247 m
robot_2 | Position=(4.32, -1.76) | Target=(4.56, -1.69) | Yaw=0.19 | Heading error=0.09 | Distance=0.25 | Left speed=1.75 | Right speed=2.00
robot_2: REACHED Waypoint 127/128 at (4.56, -1.69) | Distance: 0.245 m
robot_2: REACHED Waypoint 128/128 at (4.81, -1.69) | Distance: 0.246 m
robot_2: A* Route Completed (128/128 waypoints reached)
robot_1 | Consensus Value: 50.0000
robot_2 | Consensus Value: 50.0000
robot_3 | Consensus Value: 50.0000
```

The Multi-Robot Navigation System was implemented in Webots using A* routes and GPS-based waypoint tracking. The console log generated from the latest run confirmed that all 3 robots completed their assigned routes. Robot 1 reached 105 out of 105 waypoints, Robot 2 reached 128 out of 128 waypoints, and Robot 3 reached 114 out of 114 waypoints. Hence, a total of 347 planned waypoints were reached within the recorded simulation run. The controller used to control each robot had a waypoint tolerance value of 0.25m, and the final waypoint distances were within this threshold. The console log also recorded turning and yielding decisions made whilst the robots navigated the environment. Ultimately, the robots demonstrated successful route completion within a single simulation run, however, the console output didn't independently establish collision-free operation, total travel distance, completion time, or repeatability across multiple simulation runs. To evaluate the efficiency and flexibility of the robots within their environment, additional runs and collision and timing measurements are needed.

Emergent Group Behavior

The combined results demonstrate coordinated behavior across the three autonomous agents. The PSO algorithm distributed 24 target waypoints evenly amongst the robots, assigning 8 targets to each agent, whilst the average consensus algorithm enabled the recorded consensus values to approach a common value of 50. During the Webots Navigation Simulation Run, all 3 robots completed their assigned routes, reaching a combined total of 347 out of 347 planned waypoints. The navigation log also recorded turning and yielding decisions as the robots moved through the shared environment. Together, these results demonstrated task distribution, agreement between the recorded agent states, and even successful execution of the assigned

routes within the tested simulation. However, since only one successful navigation run was documented and the recorded consensus data had already begun close to the final value, further experiments are needed in order to establish the repeatability of the group behavior and verify the change in consensus values.

Discussion and Ethical Considerations

Component A

Trade-offs Between Perception Accuracy, Planning Optimality and Control Responsiveness

Perception, path planning and motion control are essential components that influence an autonomous mobile robot's performance in its operating environment. Although each component heavily contributes to successful navigation, improving a single one doesn't necessarily improve the performance tied to the entire system. ***(Mlejnek, 2026) (Rakhmatillaev, Bucinskas and Kabulov, 2025)***

Perception accuracy involves influencing the robot's ability to identify obstacles and even interpret its surroundings. In Component A, LiDAR measurements and camera-based edge detections provided environmental information that complemented each other. LiDAR supplied distance measurements, whilst the camera processed identified image edges. However, sensor noise, invalid measurements and the limitations of threshold-based sensor fusion could result in incorrect obstacle assessments. Obstacle detection can improve safety but can also restrict the robot's available routes, whereas missed obstacles tend to lead to unsafe navigation routes. Perception heavily involves balancing reliable obstacle detection with the need to avoid unnecessary restrictions on movement. ***(Mlejnek, 2026)***

Planning Optimality involves finding a suitable route whilst also considering the path length and computational efficiency. The results demonstrated that Dijkstra's algorithm and A* generated paths of equal length, 7.80m, whilst A* explored 127 nodes compared with 258 nodes for Dijkstra. This demonstrated that A* reduced the search effort without having to increase the planned path length in the tested environment. However, an optimal path on an occupancy grid isn't always the safest or most efficient route within dynamic environments. Any changes to obstacle positions, sensor information, and the robot's physical movement can result in the actual trajectory to differ from the planned route. The measure of efficiency tied to path planning needs to consider various aspects such as clearance of obstacles, reliability attached to routes and even the robot's ability to follow the path effectively in order to determine how efficient each path is. ***(Arora et al., 2026)***

Control Responsiveness is an aspect associated with the autonomous robot that helps to determine the efficiency and effectiveness of the robot when following its waypoints. Its effectiveness is measured by how quickly and accurately it reaches each waypoint. The PID experiments helped to demonstrate that gain adjustments affected position and heading accuracy differently. The baseline setup achieved a mean absolute

position error of 0.0362m, which decreased to 0.0165m in Experiment 2. However, the mean absolute heading error rose from approximately 0.0283 rad to 0.0483 rad. These results highlight the trade-off between enhancing positional accuracy and preserving precise orientation. More responsive control can help the robot correct deviations quickly, but excessive correction can create unstable or irregular movement. The recorded results didn't establish controller stability, which highlighted the importance of evaluating both tracking accuracy and dynamic behavior. ***(Rakhmatillaev, Bucinskas and Kabulov, 2025)***

Effective navigation requires a balance between the factors associated with the autonomous mobile robot such as perception, planning and control. By having accurate perception results, efficient path finding and planning, and responsive control, the robot achieves an effective navigation path. A computationally efficient path planner cannot compensate for incorrect sensor information, while accurate perception and an optimal planned route don't guarantee successful navigation even if the controller cannot track the route reliably. These finding ultimately emphasize the importance of measuring the performance of the navigation system instead of having to test every single component of the system independently. ***(Mlejnek, 2026) (Rakhmatillaev, Bucinskas and Kabulov, 2025)***

Practical and Ethical Considerations of Autonomous Navigation in Shared Human Environments

In environments where humans occupy the space and autonomous mobile robots are implemented, there are several ethical challenges which can arise that might not be found in the Webots simulation. Even though Component A demonstrated successful navigation within the specified goal tolerance, its results were obtained in a simulated environment and cannot establish that the system is safe to be deployed in a real-world environment. ***(Adesiji et al., 2025)***

From a practical perspective, autonomous robots must be able to detect people and obstacles reliably, respond fast and appropriately to any form of unexpected movement and even maintain enough clearance when navigation around narrow or crowded environments. There are various limitations tied to the autonomous robots, such as faulty sensors which end up producing delayed responses and even weird behavior expressed by the robots when tracing their actual trajectory when following the planned route. All of this presents a risk of collisions occurring; hence, the robot needs to be able to safely react and make decisions without incurring any collisions. ***(Adesiji et al., 2025)***

Another ethical responsibility revolves around safety, since robots operate in environments which are shared with people. This means that robots need to prioritize preventing harm to people rather than focusing on trying

to reach their destination in the shortest way possible. The robot's behavior needs to remain predictable and must also be able to reduce in speed or come to a complete halt when safety measures aren't met. Decisions revolving around obstacle avoidance shouldn't place people at unnecessary risk just for the sake of preserving path optimality or reducing travel time. ***(Adesiji et al., 2025) (Mlejnek, 2026)***

Another ethical responsibility is accountability. This is because if the robot causes damage or harm to someone, responsibility needs to be taken by the developers, operators or organizations responsible for deploying the system. Testing, maintenance, monitoring and documentation should be emphasized to identify and prevent faults from occurring and understanding the logic behind the robot's decisions. If cameras, or other sensors collect information about people, privacy needs to also be protected by limiting the data collection, storage and access to what is necessary to process the robot's operation and execute tasks. ***(Fernández-Becerra et al., 2025) (Republic of South Africa, 2013)***

Finally, autonomous systems need to also be evaluated for accessibility and fairness since a robot shouldn't assume that every person can move or react in the same way. The navigation behavior tied to each robot should account for people with mobile impairments or movements and not obstruct shared pathways. These considerations help to reinforce the need to design autonomous navigation around human safety and usability rather than technical performance alone. ***(Malik, 2025)***

Component B

Comparing Centralized vs Distributed Decision-Making Approaches

Planning for environments consisting of multiple robots involves coordinating them to achieve a related set of objectives. To perform this, mechanisms are needed to ensure robots can communicate with each other, share knowledge, be allocated tasks and even coordinate actions. ***(GeeksForGeeks, 2025c)***

In a centralized decision-making architecture, a single controller ends up making the decisions for all robots. It does this by collecting information from the system and can help simplify coordination by transmitting this information to each robot to help them when navigating their environment and executing tasks. However, since the central controller becomes a bottleneck as the number of robots increases, its failure can affect the operation of the entire system. Contrast to this, distributed decision-making enables individual agents to make decisions using available information while communicating and coordinating with other agents. This approach ends up distributing decision-making responsibilities, although effective coordination can become more complex as well as overhead from communication increasing. ***(GeeksForGeeks, 2025c)***

Component B is implemented using a distributed multi-robot approach in which three autonomous robots operate using individual instances of the same Python controller. The robots end up exchanging information through Webots Emitter and Receiver devices, whilst the average consensus algorithm supports agreement between recorded agent states. PSO is used to allocate target waypoints to each robot, and the robots follow their assigned routes using A* path planning and GPS-based waypoint tracking. The neural network aids in classifying the four navigation actions made by the robot. ***(GeeksForGeeks, 2025c)***

The Webots Simulation run which was recorded demonstrated that all three robots completed their assigned routes, reaching 347 out of 347 planned waypoints. Hence, this provides evidence that the implemented system successfully executed its route. However, a separate centralized implementation wasn't evaluated using the same environment and waypoint targets. This means that the results don't establish that decision-making is better than centralized decision-making in terms of completion time, travel distance, or collision avoidance. ***(GeeksForGeeks, 2025c)***

Bias and Reliability Risks

The reliability of the proposed multi-agent robotic system heavily depends on the quality of sensor information, simulation assumptions, and training data. Within this project, the neural network was heavily trained using synthetic, rule-labelled examples rather than relying on a comprehensive dataset collected from a real-world environment. The model achieved a test accuracy of 98.17%, with a YIELD class F1-score of 84.03%, but these results didn't at all guarantee reliable behavior under unfamiliar conditions. The overall accuracy can conceal weaker performance for particular actions, which makes class-specific metric important when needing to evaluate safety-related decisions. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

Synthetic training data might not represent all the various situations encountered by physical robots, such as sensor noise, uneven obstacle shapes as well as uneven surfaces and unexpected interactions with people or other robots. In a similar manner, GPS and IMU measurements in simulations may not be consistent than measurements obtained from real hardware. Such differences in measurements can create gaps or lead the system to produce unreliable results when transitioning the system to a physical deployment. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

The use of synthetic data also helps to avoid the need to collect personal information to ensure privacy but can very much limit the training process when being trained on examples. Reliability needs to therefore be assessed using a variety of test scenarios, which include sensor disturbances, unfamiliar obstacles, and even imbalanced action classes. Testing needs to involve the analysis of unsafe decisions which were predicted to be safe but ended up leading to a collision. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

Transparency and Explainability

The system combines algorithms with a neural-network classifier. Algorithms such as A* and PSO have identifiable objectives and procedures, which mark their behavior as being easier to inspect than the internal decision process behind a trained neural network. Even though the classifier can select navigation actions based on its inputs, the learned parameters can't provide a straight explanation as to why a certain action was chosen and executed. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

This will create an accountability challenge, since the robot can end up selecting an unsafe turning action or failing to yield. Such cases require for an operator to understand why the decision was made or chosen. Hence, the model's overall accuracy cannot be used alone to explain an individual prediction or even establish that the prediction was safe or correct in a specific situation. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

A practical mitigation is to record the sensor values supplied to the classifier, along with the predicted action, the robot's resulting movement and any relevant communication or navigation events. Comparing the classifier's decisions with those made by the rule-based expert which produced the synthetic labels can help identify any disagreements and unexpected behavior patterns. When making these comparisons, they can be treated as diagnostic evidence rather than proof to showcase the expert being always correct.

For deployment purposes, the system needs to provide interpretable event logs and clear reasoning for any safety procedures to be followed when possible. Human operators need to also accept responsibility for monitoring and investigating the system for decisions which proved to be harmful to the environment rather than ignoring it. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

Safety-Critical Decisions in Human-Shared Environments

A system such as this introduces several safety risks since each of these robots found within the swarm can move and make decisions independently and simultaneously. A navigation error made by one robot can cause a collision, obstruct another robot's route or even interfere with the team's task allocation. Any form of communication delays or conflicting local decisions can also further increase these risks. ***(Schirn, 2023)***

The project's neural network navigation model is particularly important because a classification error can translate directly into physical movement. The YIELD class F1-score of around 84.03% indicates that the model's performance for yielding decisions requires specific attention, even though the overall test accuracy is high. In a real-world environment, any incorrect decision made can result in the robot moving forward instead of yielding and presenting a threat or endangerment to other people, equipment or other robots.

The current Webots simulation provides a controlled environment for evaluating robot behavior, but successful simulated navigation doesn't establish that the system is at all safe around people. Before being physically deployed, it is vital that various checks are performed to ensure that the system accounts for all types of scenarios and handles them correctly. An example of this is a safe-stop mechanism that reduces the speed of the robots or stops them completely when sensing obstacles in the environment. ***(Schirn, 2023)***

In a real-world environment, risk assessment and risk reduction are central to the industrial robot safety standards laid out in ISO 10218. The application of these standards depends on the robot and deployment context, so it should be used as a safety guidance especially for a swarm environment. ***(Schirn, 2023)***

Fault Tolerance, Robustness and Failure Mitigation

Coordination of multiple robots heavily depends on them exchanging information and executing their assigned tasks. If a robot ends up stopping operation, sending incorrect information or even losing communication, the remaining agents may work with incomplete or outdated state estimates. Consensus convergence in normal conditions cannot, by itself, demonstrate resilience to these failures. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

For this Component, the robots used communication to support consensus and coordination, whilst PSO allocated targets and A* planned routes. A failure found within one component can result in it affecting other components within the robotic system. Examples of this include a communication failure which affects the coordination behavior of the robots, or a scenario where an invalid route can prevent a robot from performing its intended task. If the robot ends up moving despite receiving invalid inputs, it can end up creating a risk to its environment. ***(Schirn, 2023)***

Fault tolerance needs to therefore be evaluated through deliberate failure tests. Such experiments which can be recommended include disabling one robot during operation, dropping or delaying communication messages, introducing noisy sensor readings and even checking how the remaining robots behave when a planned route or required model file is unavailable. Outcomes need to be measured using task completion, collision events, recovery time and even the ability of the remaining robots to continue safely. These are proposed tests unless their results have been recorded in the project.

Appropriate mitigations include communication timeouts, validation of received messages, detection of unresponsive agents, safe stopping when critical inputs are missing, and even controlled redistribution of unfinished tasks. The system needs to avoid assuming that any missing information means a robot is safe or that every received message is correct. Failure logs can help to provide evidence of robustness than relying on successful implementations of the simulation. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

Responsible AI and Robotics in South Africa

Responsible deployment requires consideration around South Africa's legal, social and infrastructure context. The Occupational Health and Safety Act establishes a framework for protecting people at work and even others who might be affected by workplace activities. Organizations which introduce autonomous robots into warehouses, factories or other workplaces need to really consider the relevant health and safety laws and regulations, including any risks created by human and robot interactions. ***(Republic of South Africa, 1993)***

Privacy is another important aspect, because if a future implementation of the system uses cameras or other sensors to collect personally identifiable information, the POPIA Act is applied. Measures need

to be taken to prevent the collection of sensitive data, and these include defining a legitimate purpose behind data collection, limiting collection and retention, protecting any sensitive or private information, and even ensuring that the personal information is processed lawfully. The current simulation's use of synthetic training data doesn't demonstrate compliance for a future real-world deployment. ***(Republic of South Africa, 2013)***

Additionally, infrastructure reliability also is important. This factor stems from the fact that within South Africa, various issues arise which can result in the improper operation of robotic systems. Such issues include power cuts, inconsistent connections, dust and even uneven surfaces which can affect robots operating in the industrial, logistics and agricultural sectors. If communication becomes unavailable, the robots need to be able to depend on their internal logic to implement basic safety measures. By implementing local-decision making, it can help to reduce dependence on consistent connectivity. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

Finally, robotic automation and its associated benefits also need to be balanced against any impact it has on people. Within unsafe environment, automation of these processes through robots can help improve performance and increase productivity but can also tend to lead to organizations needing to consider training sessions for workers and changes to job roles. Human oversight, clear accountability and transparent operational procedures are still important alongside technical performance. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023)***

To conclude, the swarm heavily demonstrates how consensus, PSO, A* and neural networks-based navigation can be combined into a simulated multi-robot system. However, good simulation results aren't sufficient for safe, fair or reliable real-world autonomy. Responsible deployment would require for broader testing, interpretable monitoring, fault-injection experiments, safety controls and even compliance with requirements and regulations set out in South Africa. ***(National Institute of Standards and Technology and U.S. Department of Commerce, 2023) (Schirn, 2023) (Republic of South Africa, 1993) (Republic of South Africa, 2013)***

Conclusion and Recommendations

Component A

Component A successfully demonstrated an autonomous mobile robot navigation system in Webots, integrating LiDAR and Camera-based perception, Dijkstra's and A* path planning, and even PID-based control. Each planning algorithm produced the following results, where the A* algorithm explored a total of 127 nodes within a path that was 7.80m, while the Dijkstra algorithm explored a total of 258 nodes for the same path which was 7.80m. This integrated system had reached its goal within the specified 0.25m tolerance and achieved a final goal error value of 0.199m along with a mean route deviation of 0.056m. PID tuning further improved position-tracking accuracy, although the heading error increased after introducing Ki control. These results ultimately demonstrate the potential behind combining multiple sensors, path planning and feedback control for autonomous navigation. However, these findings are limited by the simulated environment, no repeatable trials, and its inability to establish stability of the controller from these recorded responses.

Recommendations for this Component revolve around focusing on evaluating the system across multiple simulation runs, as well as a variety of obstacle configurations and even different sensor conditions. In addition to this, the trade-offs tied to the position and heading accuracy need to be tested, as well as tuning the PID gains. Finally, obstacle detection and avoidance mechanisms need to be inspected under real-world conditions.

Component B

Component B successfully demonstrated a multi-robot simulation in Webots involving three autonomous agents. The implementation combined average consensus, Particle Swarm Optimization, A* Path Planning and Neural Network-based navigation decisions. The simulation run which was recorded achieved completion across all 347 planned waypoints, whilst the neural network obtained a test accuracy of 98.17% on synthetically labelled data. These results each demonstrate the potential behind combining swarm intelligence and machine learning for coordinated robotic navigation. However, each of these findings are limited by the synthetic training data and the absence of repeated trials and a direct comparison with centralized architecture.

Future work should focus on evaluating the system across multiple simulation runs, varying sensor conditions, obstacle configurations and even robot interactions. The neural network also needs to be tested using representative data, with particular attention being drawn to incorrect YIELD predictions. Fault-tolerance and

safety mechanisms also need to be implemented and evaluated based on their performance. Finally, centralized and distributed architecture needs to be compared to understand the performance of the simulation and determine which of the two environments produces real-world results.

Bibliography

Adesiji, A.D. *et al.* (2025) 'Safety considerations in deployment of robotic systems - A systematic review,' *Journal of Field Robotics* [Preprint]. https://doi.org/10.1002/rob.70022.

Arora, J.K. *et al.* (2026) *Balancing Safety and Optimality in Robot Path Planning: Algorithm and Metric*. https://arxiv.org/html/2505.23197 (Accessed: October 23, 2026).

Danu Jaya Saputro (2022) *Using Webots as a Learning Platform of Robotics Simulation*. https://medium.com/@danujaya04/using-webots-as-a-learning-platform-of-robotics-simulation-4c9716e3aa3d (Accessed: October 23, 2026).

Fernández-Becerra, L. *et al.* (2025) 'Improving accountability and explainability in robots through encryption, large language models, and visual language models,' *Logic Journal of the IGPL*, 34(1). https://doi.org/10.1093/jigpal/jzaf016.

GeeksForGeeks (2025a) *Consensus Clustering*. https://www.geeksforgeeks.org/machine-learning/consensus-clustering/ (Accessed: October 23, 2026).

GeeksForGeeks (2025b) *Dijkstra's shortest path algorithm in Python*. https://www.geeksforgeeks.org/python/python-program-for-dijkstras-shortest-path-algorithm-greedy-algo-7/ (Accessed: October 23, 2026).

GeeksForGeeks (2025c) *Multiagent Planning in AI*. https://www.geeksforgeeks.org/artificial-intelligence/multiagent-planning-in-ai/ (Accessed: October 23, 2026).

GeeksForGeeks (2026a) *A* Search Algorithm in Python*. https://www.geeksforgeeks.org/python/a-search-algorithm-in-python/ (Accessed: October 23, 2026).

GeeksForGeeks (2026b) *OS Path Module in Python*. https://www.geeksforgeeks.org/python/os-path-module-python/ (Accessed: October 23, 2026).

GeeksForGeeks (2026c) *Particle Swarm Optimization (PSO)*. https://www.geeksforgeeks.org/machine-learning/particle-swarm-optimization-pso-an-overview (Accessed: October 23, 2026).

Malik, R. (2025) *How Robotics is Transforming Accessibility for Millions*. https://www.howtogeek.com/how-robotics-is-transforming-accessibility-for-millions/ (Accessed: October 23, 2026).

MATLAB (no date) *Differential Drive Kinematic Model*. https://www.mathworks.com/help/robotics/ref/differentialdrivekinematicmodel.html (Accessed: October 23, 2026).

matplotlib (no date) *matplotlib.pyplot.imshow*. https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.imshow.html (Accessed: October 23, 2026).

Mlejnek, M. (2026) *When Robot Safety Becomes a Perception Problem*. https://framos.com/articles/when-

robot-safety-becomes-a-perception-problem/ (Accessed: October 23, 2026).
National Institute of Standards and Technology and U.S. Department of Commerce (2023) *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.AI.100-1.
NI (2025) *The PID Controller & Theory Explained*. https://www.ni.com/en/shop/labview/pid-theory-explained.html?srsltid=AYCX9vIrZrvM9NKjzSGcLlezXPFvBjUHZas8wklfg8rEYUR7bnbAG2cd (Accessed: October 23, 2026).
Rakhmatillaev, J., Bucinskas, V. and Kabulov, N. (2025) 'An integrative review of control strategies in robotics,' *Robotic Systems and Applications*, 5(2). https://doi.org/10.21595/rsa.2025.25014.
Republic of South Africa (1993) *Occupational Health and Safety Act 85 of 1993*, *www.gov.za*. Cape Town, Western Cape, South Africa: www.gov.za.
https://www.gov.za/sites/default/files/gcis_document/201409/act85of1993.pdf (Accessed: October 23, 2026).
Republic of South Africa (2013) *Protection of Personal Information Act 4 of 2013*. www.justice.gov.za. https://www.justice.gov.za/legislation/acts/2013-004.pdf (Accessed: October 23, 2026).
Schirn, A. (2023) *ISO 10218-1:2025—Robots And Robotic Devices Safety*. https://blog.ansi.org/ansi/iso-10218-1-2025-robots-and-robotic-devices-safety/ (Accessed: October 23, 2026).